AutoMarts - Final Independent Verification Report

Date: 2026-10-31 (Update)
Branch: claude/elated-mestorf-77dc0f
Commit: 3d415a6
PR: https://github.com/exnesssjaffri-sketch/parts/pull/2

1. Executive Summary

AutoMarts is a full-stack auto parts e-commerce application built with Express 5 (TypeScript), React 18 (Vite), SQLite (sql.js WASM), and Supabase. This update documents the security hardening completed on 2026-10-31, including cross-user challenge token attack prevention, Google OAuth MFA enforcement, 18 additional regression tests (68 total), Supabase migration 003, and comprehensive documentation updates.

2. Security Fixes Applied

P0 - MFA Challenge Token Architecture

The MFA challenge flow was rearchitected with a dedicated challenges Map that provides independent TTL (2 min), max attempt limits (5), and single-use semantics. Additionally, the challenge token is now bound to the user who requested it, preventing cross-user token replay attacks.

P1 - High Severity Fixes

Finding	Fix
Transaction atomicity broken	All queries inside transactions now use prepare(sql, db).run() instead of global qRun()
Protected routes exposed	All sensitive routes wrapped with ProtectedRoute component
HTML sanitization broken	Method chain restored in sanitizeHtml()
Vehicle finder API mismatch	Frontend and backend aligned on numeric IDs
Duplicate SearchBar in Header	Removed locally defined SearchBar
Login.tsx TDZ bug	Moved useState declarations before useEffect

SEC-11 - Cross-User Challenge Token Attack (Fixed)

Before: consumeChallengeToken() atomically deleted the token on first use but did not verify the token's userId matched the authenticating user. An attacker who obtained User A's challenge token could use it with User B's credentials to authenticate as User B.
After: New consumeChallengeTokenForUser(token, userId) validates ownership before deletion. Mismatched-user attempts count toward the 5-attempt cap without consuming the token.

SEC-12 - Google OAuth MFA Bypass (Fixed)

Before: After a successful Google OAuth exchange, if the user had MFA enabled the code created a session with mfaPending: true and stored the challenge token in req.__mfaChallengeToken (request-local), which was lost. The client would receive a session without MFA verification.
After: Uses the same generateChallengeToken(userId) + body-passed token pattern as password login. The frontend /mfa/verify flow already supports this.

3. Test Results

Total: 68 tests passing across 6 test files (up from 50 across 5 files).

Test File	Count	Coverage
auth.test.ts	20	Password hashing, user CRUD, session management, MFA functions, challe
auth-http-mfa.test.ts	12	HTTP-level MFA regression: login flow, token expiry, single-use, cross-user,
sanitize-html.test.ts	11	Script injection, event handlers, mixed HTML payloads
schema.test.ts	9	Database schema integrity, index existence
audit.test.ts	7	Audit logging core functions, IP extraction
transaction-rollback.test.ts	3	Commit on success, rollback on failure, no partial state

Build Verification:
* Backend TypeScript: PASS - 0 errors
* Frontend build: PASS - 2029 modules
* Secret scan: PASS - no real credentials found

4. HTTP-Level MFA Regression Tests (New)

All 12 scenarios exercise the real Express app on an ephemeral localhost port, including the real Google OAuth callback endpoint:

#	Scenario	Status
1	Normal password login for non-MFA user returns valid session	PASS
2	MFA-enabled login returns mfa_required + challenge_token, no session cookie	PASS
3	Valid challenge_token + valid TOTP creates authenticated session	PASS
4	Invalid TOTP rejected; single-use token consumed even on failure	PASS
5	Expired challenge token rejected (deterministic via test helper)	PASS
6	Consumed challenge token cannot be reused	PASS
7	User A challenge token cannot authenticate User B	PASS
8	Challenge token cannot access protected endpoints (/api/auth/me)	PASS
9	Five failed attempts invalidate the challenge	PASS
10	Google OAuth follows same MFA rules (non-MFA -> session; MFA -> challenge_token)	PASS
11	Non-MFA user not forced through MFA flow	PASS
12	Logout invalidates session and blocks subsequent /api/auth/me	PASS

5. Supabase RLS Security (Migration 003)

Supabase Security Advisor identified issues in the PostgreSQL instance. Migration 003 is ready but NOT applied to the live database.

Severity	Finding	Tables/Details
ERROR	RLS Disabled on Public Tables	users, orders, cart_items, addresses, coupons, audit_logs, wishlist_i
WARN	SECURITY DEFINER executable by anon	rls_auto_enable()
WARN	SECURITY DEFINER executable by authenticated	rls_auto_enable()

INFO	RLS Enabled but No Policy	schema_version
INFO	Unindexed Foreign Keys	5 foreign keys without indexes

Resolution: Migration 003 (supabase/migrations/003_rls_security.sql) enables RLS on all 10 tables with per-role policies, revokes rls_auto_enable() EXECUTE, and handles schema_version dead lock. MUST BE APPLIED MANUALLY in the Supabase SQL editor before any direct PostgREST exposure.

6. Git Diff Summary

32 files changed: 1834 insertions(+), 160 deletions(-)

File	Change
backend/src/auth/index.ts	Added generateChallengeToken(), consumeChallengeTokenForUser(), __debugExpireChalle
backend/src/index.ts	Made startServer() conditional for test compatibility
backend/src/routes/auth.ts	Fixed MFA challenge endpoint, added consumeChallengeTokenForUser import, OAuth MFA c
backend/test/auth-http-mfa.test.ts	NEW - 12 HTTP-level MFA regression tests
backend/test/auth.test.ts	20 tests (password, user CRUD, session, MFA, challenge flow)
backend/test/sanitize-html.test.ts	NEW - 11 XSS regression tests
backend/test/transaction-rollback.test.ts	NEW - 3 transaction atomicity tests
frontend/src/App.tsx	ProtectedRoute wrapper for authenticated/admin routes
frontend/src/context/AuthContext.tsx	mfaChallengeToken support
frontend/src/pages/Login.tsx	TDZ fix, MFA flow handling
frontend/src/pages/MFALogin.tsx	Challenge token consumption
frontend/src/components/Header.tsx	Removed duplicate SearchBar
frontend/src/components/VehicleFinder.tsx	Numeric ID alignment
supabase/migrations/003_rls_security.sql	NEW - RLS policies for 10 tables

7. Documentation

File	Description
docs/ARCHITECTURE.md	System architecture documentation
docs/API_DOCUMENTATION.md	Full REST API reference
docs/DATABASE_DOCUMENTATION.md	Database schema and table reference
docs/SECURITY_AUDIT.md	Security audit findings and Supabase scan results
docs/TESTING_REPORT.md	Test suite overview and coverage (68 tests)
docs/PROJECT_DESIGN_DECISIONS.md	Architecture decision records
docs/PROJECT_OVERVIEW.md	Project overview and tech stack
docs/PROJECT_STATUS.md	Current project status and deployment blockers
CHANGELOG.md	Change log with all fixes and additions
supabase/migrations/003_rls_security.sql	RLS migration (pending manual application)

8. Pending Actions (Manual)

1. Apply migration 003 in Supabase SQL editor (10-table RLS enablement + policies). See supabase/migrations/003_rls_security.sql.
2. Set production secrets (JWT_SECRET, SESSION_SECRET, SUPABASE_URL, SUPABASE_ANON_KEY).
3. Configure CORS for production domain (restrict ALLOWED_ORIGINS).
4. Decide on MFA challenge store persistence if multi-instance/serverless deployment is planned.

9. Conclusion

All critical (P0) and high (P1) security issues have been resolved. The MFA challenge architecture now uses a dedicated single-use token store with user-binding, independent TTL, and attempt limits. Google OAuth follows the same MFA rules as password login. All 68 tests pass. Backend TypeScript compiles clean. Frontend builds successfully.
Repository: https://github.com/exnesssjaffri-sketch/parts Branch: claude/elated-mestorf-77dc0f PR: https://github.com/exnesssjaffri-sketch/parts/pull/2 Commit: 3d415a6
NOT production-ready until: migration 003 is applied and production secrets are configured.